

Relatório de Programação Concorrente 25/26

Miguel Gonçalves A108478

João Taveira A108559

José Novais A108395

Luís Lopes A108396

Professor: Paulo Sérgio Soares Almeida

16 de maio de 2026

Índice

1	Introdução	3
2	Desenvolvimento	4
2.1	Análise do Problema	4
2.2	Solução Implementada	4
3	Implementação	5
3.1	Estrutura do Projeto	5
3.2	Comunicação Cliente-Servidor	5
3.3	Gestão Concorrente das Partidas	5
4	Conclusão	6

1 Introdução

No âmbito da unidade curricular de Programação Concorrente foi proposto o desenvolvimento de um jogo multijogador em tempo real. O projeto tem como principal objetivo aplicar conceitos relacionados com concorrência, comunicação em rede e sincronização entre processos, recorrendo às linguagens Erlang e Java. O jogo desenvolvido permite a interação simultânea de vários jogadores num ambiente bidimensional, onde cada utilizador controla um avatar capaz de se movimentar e interagir com outros elementos do jogo. Ao longo do desenvolvimento foram abordados diferentes desafios associados à gestão de múltiplos clientes, troca contínua de informação e atualização consistente do estado do jogo. Este relatório apresenta a análise do problema proposto, a solução desenvolvida, a estrutura do projeto e as principais conclusões obtidas durante a implementação.

2 Desenvolvimento

2.1 Análise do Problema

O principal desafio do projeto consiste em garantir o funcionamento correto de um jogo multijogador em tempo real, assegurando que vários jogadores consigam interagir simultaneamente sem comprometer a consistência do estado do jogo. O sistema deve gerir autenticação de utilizadores, criação de partidas, movimentação dos jogadores, colisões, capturas e atualização das pontuações. Além disso, é necessário garantir que a informação recebida por cada cliente se mantém sincronizada ao longo da partida. Relativamente aos requisitos funcionais, o sistema deve permitir:

- Registo, autenticação e remoção de utilizadores;
- Criação de partidas com 3 a 4 jogadores;
- Movimentação dos jogadores num espaço 2D;
- Interação entre jogadores e objetos do cenário;
- Sistema de capturas e pontuação;
- Atualização contínua do estado do jogo.

Quanto aos requisitos não funcionais, destacam-se:

- Suporte para múltiplas partidas em simultâneo;
- Comunicação eficiente entre cliente e servidor;
- Capacidade de processamento concorrente;
- Sincronização do estado do jogo entre jogadores.

2.2 Solução Implementada

Para responder aos requisitos do projeto foi adotada uma arquitetura cliente-servidor. O servidor ficou responsável pela execução da lógica do jogo e gestão das partidas, enquanto o cliente trata da interface gráfica e interação do utilizador. A implementação do servidor em Erlang permitiu utilizar concorrência baseada em processos leves, facilitando a execução simultânea de várias tarefas independentes, como gestão de clientes, matchmaking e partidas ativas. A comunicação entre cliente e servidor é realizada através de sockets TCP, possibilitando a troca contínua de mensagens relacionadas com ações dos jogadores e atualização do estado do jogo. O cliente foi desenvolvido em Java com recurso à biblioteca Processing, utilizada para representar graficamente o ambiente de jogo e os diferentes elementos presentes na partida. Esta abordagem permitiu separar claramente a lógica da aplicação da componente gráfica, tornando o sistema mais organizado e modular.

3 Implementação

3.1 Estrutura do Projeto

O projeto encontra-se dividido em duas componentes principais: servidor e cliente. A componente do servidor está organizada em vários módulos independentes, cada um responsável por uma funcionalidade específica do sistema:

- **tcp_server**: aceita novas ligações TCP;
- **client_handler**: gere a comunicação individual com cada cliente;
- **matchmaker**: organiza os jogadores em filas de espera e cria partidas;
- **game_session**: executa a lógica de cada partida;
- **top_manager**: mantém o registo das pontuações.

O cliente encontra-se dividido entre componentes responsáveis pela interface gráfica, processamento do input do utilizador e comunicação com o servidor. A interface gráfica possui diferentes estados da aplicação, incluindo autenticação, fila de espera e jogo em execução.

3.2 Comunicação Cliente-Servidor

A troca de informação entre cliente e servidor é realizada de forma contínua ao longo da partida. Sempre que o jogador executa uma ação de movimento, o cliente envia um comando correspondente ao servidor, indicando as teclas atualmente pressionadas. Após processar todas as ações dos jogadores, o servidor atualiza o estado da partida e envia para cada cliente informação relativa às posições dos jogadores, objetos presentes no mapa, massas e pontuações atuais. Para reduzir complexidade na comunicação, foi utilizado um protocolo textual simples, permitindo representar facilmente os diferentes tipos de mensagens trocadas entre as duas componentes do sistema.

3.3 Gestão Concorrente das Partidas

Cada partida possui um ciclo de atualização independente, responsável por executar periodicamente a simulação do jogo. Durante cada atualização são calculados movimentos, colisões, capturas e alterações de pontuação. A separação das partidas em processos distintos permitiu executar vários jogos em simultâneo sem interferência entre sessões. Desta forma, operações realizadas numa partida não afetam o estado das restantes. A utilização de concorrência em Erlang simplificou também a gestão dos jogadores ligados ao servidor, permitindo tratar múltiplas conexões e eventos em paralelo.

4 Conclusão

Como conclusão deste trabalho, consideramos que foi uma forma eficaz de aplicar os conteúdos lecionados nas aulas práticas e teóricas, nomeadamente a utilização de barreiras no processo de matchmaking. O resultado final correspondeu às nossas expectativas, refletindo uma implementação sólida dos conceitos abordados.

Adicionalmente, conseguimos implementar concorrência em Java com recurso a threads e mecanismos de sincronização, como locks, o que contribuiu para uma melhor organização e controlo das diferentes operações do sistema, incluindo no Processing, através da utilização de uma thread dedicada a cada tipo de operação. Por fim estamos satisfeitos com o trabalho desenvolvido e com os resultados alcançados.